

Materia: Seguridad en el Desarrollo de Aplicaciones

Unidad: Vulnerabilidades Web en Aplicaciones .NET Core

Práctica: SEGG-U1-P2F – Remediación de Vulnerabilidades OWASP

Alumno: Saul Noe Perez Aguilera

Fecha: 05 de junio de 2026

Proyecto: VulnerableApp

Rama de trabajo: secure

Objetivo

Aplicar mecanismos de seguridad para corregir las vulnerabilidades identificadas previamente en la aplicación VulnerableApp, implementando controles que mitiguen los riesgos asociados a SQL Injection, autenticación insegura, almacenamiento inseguro de contraseñas, Cross-Site Scripting (XSS) e Insecure Direct Object Reference (IDOR).

Introducción

La seguridad en aplicaciones web es un aspecto fundamental durante el ciclo de desarrollo de software. Las vulnerabilidades presentes en una aplicación pueden comprometer la confidencialidad, integridad y disponibilidad de la información. En esta práctica se implementaron medidas correctivas basadas en las recomendaciones de OWASP para mitigar diversas vulnerabilidades detectadas en una aplicación desarrollada con ASP.NET Core.

El objetivo principal fue fortalecer la seguridad de la aplicación mediante la implementación de buenas prácticas de programación segura y mecanismos adecuados de validación, autenticación y autorización.

Desarrollo

Se creó una nueva rama denominada **secure** con el propósito de realizar todas las modificaciones de seguridad sin afectar la versión vulnerable de la aplicación. Esta estrategia permitió mantener un control adecuado de cambios y facilitar la comparación entre ambas versiones.

Se identificó que las consultas podrían ser vulnerables a inyecciones SQL cuando los datos ingresados por el usuario eran utilizados directamente para construir consultas.

Para solucionar este problema se reemplazó la construcción dinámica de consultas por el uso de Entity Framework Core y expresiones LINQ, permitiendo que los parámetros fueran procesados de forma segura por el framework.

Esta modificación eliminó la posibilidad de manipular consultas mediante entradas maliciosas y redujo significativamente el riesgo de acceso no autorizado a la información almacenada en la base de datos.

Se detectó que las contraseñas eran almacenadas sin protección adecuada, lo que representaba un riesgo crítico en caso de acceso no autorizado a la base de datos.

Como medida de mitigación se implementó el algoritmo BCrypt para generar hashes seguros antes de almacenar las contraseñas. Este mecanismo incorpora salting automático y un costo computacional configurable que dificulta los ataques de fuerza bruta y diccionario.

Gracias a esta implementación, las contraseñas ya no son almacenadas en texto plano y su recuperación directa resulta prácticamente imposible.

El mecanismo de autenticación fue modificado para validar las credenciales utilizando los hashes generados por BCrypt.

Durante el proceso de inicio de sesión, la contraseña proporcionada por el usuario se compara contra el hash almacenado mediante funciones especializadas de verificación. De esta manera se garantiza que las credenciales sean validadas de forma segura sin exponer información sensible.

La implementación mejora significativamente la protección del proceso de autenticación y evita prácticas inseguras relacionadas con el manejo de contraseñas.

Se identificó la posibilidad de ejecutar código JavaScript mediante campos de entrada controlados por el usuario.

Para corregir esta vulnerabilidad se implementó la codificación de salida utilizando funciones de escape HTML proporcionadas por ASP.NET Core. Con esta medida, cualquier contenido ingresado por el usuario es mostrado únicamente como texto y no puede ser interpretado por el navegador como código ejecutable.

La corrección previene la ejecución de scripts maliciosos y protege tanto a la aplicación como a los usuarios que interactúan con ella.

Se observó que un usuario podía intentar acceder a información perteneciente a otros usuarios modificando directamente los identificadores enviados en las solicitudes.

Para mitigar esta vulnerabilidad se implementaron validaciones de autorización que verifican que el identificador solicitado corresponda al usuario autenticado en la sesión actual. En caso contrario, el sistema devuelve una respuesta de acceso denegado.

Adicionalmente, las respuestas de la API fueron limitadas para evitar la exposición de información sensible como contraseñas o hashes de autenticación.

Resultados Obtenidos

Después de aplicar las correcciones correspondientes, la aplicación presentó una mejora significativa en sus mecanismos de seguridad. Las vulnerabilidades identificadas durante las prácticas anteriores fueron mitigadas mediante controles alineados con las recomendaciones de OWASP y las buenas prácticas de desarrollo seguro en ASP.NET Core.

Las pruebas realizadas demostraron que los intentos de explotación previamente exitosos dejaron de funcionar, validando la efectividad de las medidas implementadas.

Tabla Comparativa Final

Vulnerabilidad	Situación Inicial	Situación Corregida
SQL Injection	Posibilidad de manipular consultas SQL	Consultas seguras mediante Entity Framework Core
Contraseñas en texto plano	Contraseñas visibles en base de datos	Contraseñas protegidas mediante BCrypt
Autenticación insegura	Comparación directa de credenciales	Verificación segura mediante hash
XSS	Ejecución de scripts en comentarios	Codificación HTML de la salida
IDOR	Acceso a información de otros usuarios	Validación de autorización basada en sesión
Exposición de datos sensibles	Posible visualización de contraseñas o hashes	Información sensible excluida de las respuestas

Conclusiones

La realización de esta práctica permitió comprender la importancia de incorporar controles de seguridad desde las etapas tempranas del desarrollo de software. Vulnerabilidades aparentemente simples pueden convertirse en riesgos graves si no son tratadas adecuadamente.

La implementación de mecanismos como consultas parametrizadas, hashing de contraseñas, validación de autorización y codificación de contenido demostró ser una estrategia efectiva para fortalecer la seguridad de la aplicación.

Asimismo, se reforzó el conocimiento sobre las vulnerabilidades más comunes incluidas dentro del OWASP Top 10 y sobre las técnicas utilizadas para mitigarlas dentro del entorno ASP.NET Core. Como resultado, la aplicación VulnerableApp cuenta ahora con una versión más segura y preparada para resistir ataques relacionados con las vulnerabilidades abordadas durante esta práctica.